

What is a Class?

A **class** in Python is a **blueprint or template** used to create objects. It defines:

- **Attributes (variables)** → data
- **Methods (functions)** → behavior

It represents a **real-world concept** like Student, Car, BankAccount, etc.

Syntax of a Class

```
class ClassName:  
    # attributes and methods
```

What is an Object?

An **object** is an **instance of a class**.

It represents a **real-world entity** and contains:

- Data (attributes)
- Functions (methods)

Example:

- Class → Student
- Object → Sachin, Rahul
-

◆ Creating a Class and Object

```
class Student:  
    def __init__(self, name, age):  
        self.name = name        # instance variable
```

```
        self.age = age

    def display(self):
        print(self.name, self.age)

# Creating object
s1 = Student("Sachin", 21)
s1.display()
```

◆ Key Points

- `__init__()` → constructor (called automatically)
 - `self` → refers to current object
 - Object is created using class name
-

◆ Class Attributes: Class vs Instance Variables

1. Instance Variable

◆ Definition

An **instance variable** is a variable that is **specific to each object**.

◆ Key Features

- Defined inside the constructor (`__init__`) using `self`
- Each object has its **own separate copy**
- Changing one object's value does **not affect others**

◆ Example

```
class Student:
    def __init__(self, name):
        self.name = name    # instance variable

s1 = Student("Sachin")
s2 = Student("Rahul")

print(s1.name)    # Sachin
print(s2.name)    # Rahul
```

👉 Each object stores its own data.

◆ 2. Class Variables

Definition

A **class variable** is a variable that is **shared among all objects of a class**.

◆ Key Features

- Defined inside class but **outside methods**
- Only **one copy** is created
- Shared by all objects

◆ Example

```
class Student:
    college = "MIT-WPU"    # class variable

    def __init__(self, name):
        self.name = name

s1 = Student("Sachin")
s2 = Student("Rahul")
```

```
print(s1.college)    # MIT-WPU
print(s2.college)    # MIT-WPU
```

◆ Difference Table

Feature	Instance Variable	Class Variable
Scope	Object-specific	Shared
Defined	Inside constructor	Inside class
Memory	Separate copy	Single copy

◆ Types of Methods in Python

An **instance method** is the most commonly used type of method in Python classes. It is designed to operate on **individual objects**, meaning it works with the data that belongs to a specific instance of the class. This is why every instance method must take `self` as its first parameter. The keyword `self` represents the current object and allows access to its attributes and other methods.

In practice, instance methods are used when the behavior depends on the unique data of each object. For example, if you have a `Student` class, each student will have different values like name and age. An instance method such as `display()` will use `self.name` and `self.age`, ensuring that the method behaves differently for each object.

◆ 1. Instance Methods

- Work with object data
- Take `self` as parameter

```
class Demo:
    def show(self):
        print("Instance method")
```

◆ 2. Class Methods

A **class method** is associated with the class itself rather than any individual object. Instead of `self`, it uses `cls` as its first parameter, which refers to the class. To define a class method, the `@classmethod` decorator is used.

Class methods are particularly useful when you need to work with **class-level data**, such as class variables that are shared among all instances. They are also commonly used for creating **alternative constructors**, where objects are initialized in different ways.

- Work with class variables
- Use `@classmethod`
- Take `cls` as parameter

```
class Demo:
    count = 0

    @classmethod
    def show_count(cls):
        print(cls.count)
```

Although class methods can be called using objects, they fundamentally do not rely on instance-specific information unless an object is explicitly passed.

◆ 3. Static Methods

A **static method** is conceptually different from both instance and class methods. It does not depend on either the object (`self`) or the class (`cls`). It behaves like a **regular function**, but it is logically grouped inside a class because it is related to that class's functionality.

Static methods are defined using the `@staticmethod` decorator. Since they do not have access to instance or class variables directly, they are mainly used for **utility or helper functions** that perform a task related to the class but do not need its internal data.

- Independent of class and object
- Use `@staticmethod`
- No `self` or `cls`

```
class Demo:
    @staticmethod
    def greet():
        print("Hello")
```

◆ Comparison of Methods

Method Type	Parameter	Access
Instance	<code>self</code>	Object data
Class	<code>cls</code>	Class data
Static	None	Utility function

Another way to understand this is through access:

- Instance methods can access both instance and class variables.
- Class methods can access only class variables directly.
- Static methods do not access either unless explicitly passed.

◆ Inheritance in Python

◆ What is Inheritance?

Inheritance allows one class to **reuse properties and methods of another class**.

👉 Promotes **code reusability**

◆ Syntax

```
class Parent:
    def show(self):
        print("Parent class")

class Child(Parent):
    def display(self):
        print("Child class")

obj = Child()
obj.show()
obj.display()
```

◆ Types of Inheritance

1. Single Inheritance

One parent → One child

```
class A:
    pass

class B(A):
    pass
```

2. Multiple Inheritance

Multiple parents → One child

```
class A:
    pass

class B:
```

```
class C(A, B):  
    pass
```

3. Multilevel Inheritance

Chain structure

```
class A:  
    pass  
  
class B(A):  
    pass  
  
class C(B):  
    pass
```

4. Hierarchical Inheritance

One parent → Multiple children

```
class A:  
    pass  
  
class B(A):  
    pass  
  
class C(A):  
    pass
```

◆ Method Overriding

Child class can **modify parent method**

```
class Parent:  
    def show(self):  
        print("Parent")  
  
class Child(Parent):
```



```
def show(self):  
    print("Child")
```

◆ **super() Function**

Used to call parent class methods

```
class Parent:  
    def __init__(self):  
        print("Parent constructor")  
  
class Child(Parent):  
    def __init__(self):  
        super().__init__()  
        print("Child constructor")
```

◆ **Advantages of OOP Concepts**

- Code reusability
 - Modularity
 - Easy maintenance
 - Real-world modeling
-

✓ **Final Short Notes (Exam Ready)**

- **Class:** Blueprint for objects
- **Object:** Instance of class
- **Instance Variable:** Object-specific data
- **Class Variable:** Shared data
- **Instance Method:** Works on object (`self`)
- **Class Method:** Works on class (`cls`)
- **Static Method:** Utility method

- **Inheritance:** Reusing code from parent class